

Lecture 2: Image Classification with Linear Classifiers

데이터 기반 접근 방식을 포함한 첫 세가지 주요 항목

무엇인지, 선형 분류와 k 최근접 이웃 알고리즘

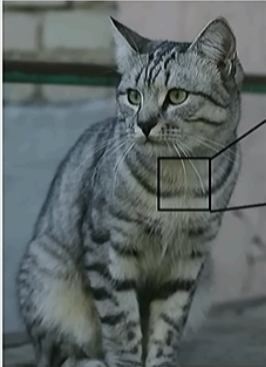
이미지 분류 : cv의 핵심 과제

- 최근접 분류기
- 선형 분류기

이미지 분류란? 이미지와 미리 정의된 라벨(강아지, 고양이 등)이 주어졌을 때 시스템이 라벨 중 하나를 할당하는 것

우리의 뇌인지 시스템이 이미지를 전반적으로 이해하고 라벨을 붙이지만 computer가 어떻게 그 과정을 거치는지는 다른 이야기이다.

The Problem: Semantic Gap



[	100	112	108	111	104	99	106	99	96	103	112	119	104	97	93	87]
[	91	98	102	106	104	79	98	103	99	105	123	136	110	105	94	85]
[	76	85	98	105	128	185	87	96	95	99	115	112	106	103	99	85]
[	99	81	81	93	120	121	127	100	95	98	102	99	96	93	101	94]
[	106	91	61	64	69	91	88	85	101	107	109	98	75	84	96	95]
[	114	188	85	55	55	69	64	54	64	87	112	129	98	74	84	91]
[	135	137	147	183	65	81	88	65	52	54	74	84	182	93	85	82]
[	128	137	144	148	189	95	86	78	62	65	63	63	68	73	86	181]
[	125	133	148	137	139	121	117	94	65	79	88	65	54	64	72	98]
[	127	125	131	147	133	127	126	131	111	96	89	75	61	64	72	84]
[	115	114	189	123	158	148	131	118	113	189	188	92	74	65	72	78]
[	89	93	98	97	188	147	131	118	113	114	113	188	186	95	77	88]
[	63	77	86	81	77	79	182	123	117	115	117	125	125	138	115	87]
[	62	65	82	89	78	71	88	181	124	126	119	181	187	114	131	119]
[	63	65	75	88	89	71	62	81	128	138	135	185	81	98	118	118]
[	87	65	71	87	186	95	69	45	76	138	126	187	92	94	185	112]
[	118	97	82	86	117	123	116	66	41	51	95	93	89	95	182	187]
[	164	146	112	88	82	128	124	184	76	48	45	66	88	181	182	189]
[	157	178	157	128	93	86	114	132	112	97	69	55	78	82	99	94]
[	138	128	134	161	139	188	189	118	121	134	114	87	65	53	69	86]
[	128	112	96	117	158	144	128	115	184	187	182	93	87	81	72	79]
[	122	187	96	86	83	112	153	149	122	189	184	75	88	187	112	99]
[	122	121	182	88	82	86	94	117	145	148	153	182	58	78	92	187]
[	122	164	148	183	71	56	78	83	93	183	119	139	182	61	69	84]]

What the computer sees

An image is a tensor of integers between [0, 255]:

e.g. 800 x 600 x 3 (3 channels RGB)

컴퓨터 : 이 이미지는 데이터 행렬로 정의되며 각 픽셀값은 0에서 255 사이의 8bit 데이터 구조, 해상도 800 * 600 이미지, RGB 채널의 이미지.

=우리가 이미지를 인식하는 방식과 기계가 이미지를 인식하는 방법의 의미론적 차이

카메라를 움직인다고 가정한다면 좌우로 움직이는 경우 고양이가 가만히 있어도 모든 픽셀 값은 변경된다. 그러면 이 모든 픽셀은 새로운 값을 갖게 되어 인간에게는 같은 물체로 인식 되더라도 컴퓨터의 관점에서는 아예 새로운 데이터이다.

조명, 방향, 재질, 광원 등에 따라 같은 물체여도 RGB 값은 다르다.

Challenges: Occlusion

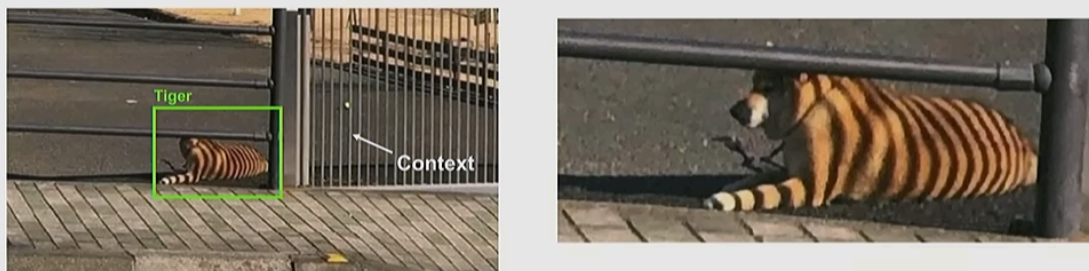


배경의 차이, 물체의 가려짐의 정도 등 위 사진처럼 인간의 뇌인지는 바로 고양이라고 보는 것을 컴퓨터 역시 고양이로 인식할 수 있게 하는 것은 어려운 과제이다

클래스 내 변동성 역시 중요하다. 고양이가 크기, 색깔, 무늬, 품종이 여러가지라는 것을 알고 그 모든 것이 고양이라고 알지만 컴퓨터는 클래스 내 변이를 인식하는 것이 어렵다.

맥락 역시 흥미로운 과제

Challenges: Context



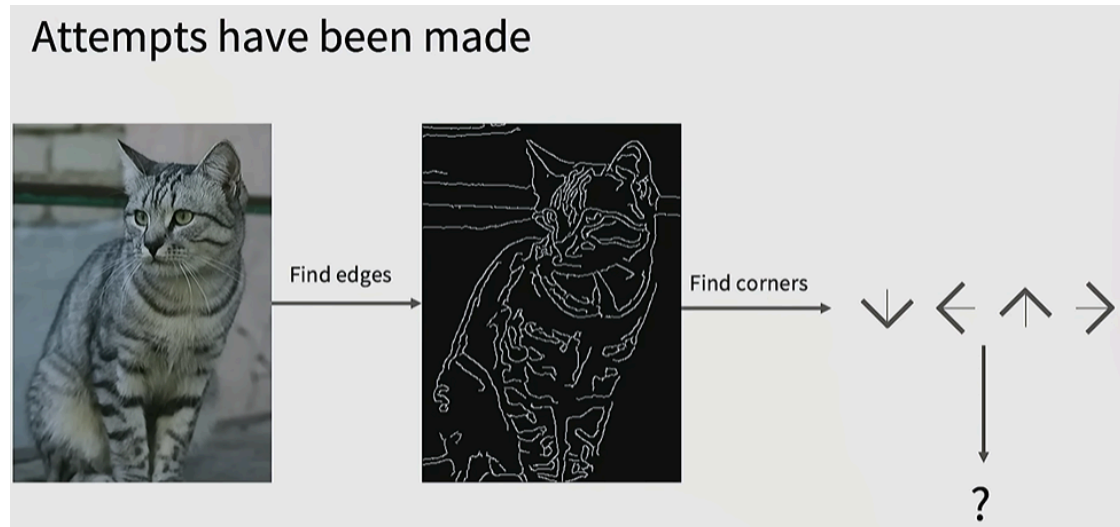
그림자 효과, 배경 등의 맥락을 이해하면 고양이라고 하겠지만 컴퓨터는 호랑이라고 분류하는 문제 발생

ImageNet과 같은 데이터 집합이나 다양한 모델들 덕분에 객체를 검증하는 기술은 성능이 꽤 좋다.

그러한 대규모 알고리즘을 구축하는 데 필요한 구성요소들

= 함수를 구현. 하지만 이미지를 분류하는 단계를 코드로 미리 정해둘 방법이 어렵기 때문에 불가능하다

객체를 인식하기 위한 알고리즘의 단계 = 엣지 검출기 기반.



이미지에서 엣지를 찾아 패턴을 만들고 모서리 같은 특징을 기반으로 출력 클래스에 매핑을 시도. 제한적인 이미지 유형에서는 어느 정도 성공적이지만 확장하는 것은 매우 어렵다

각각의 논리를 찾는 것 자체만으로도 상당한 노력이 필요하다

= 이미지 분류의 논리와 절차를 만드는 데 기반한 알고리즘은 그닥 성공적이지 못했다

= 머신러닝이 해결법

→

= 데이터 기반 접근 방식을 제시

= 3단계 프로세스 정의

1. 이미지와 해당 레이블로 구성된 데이터 세트 수집
2. 머신러닝 알고리즘을 사용하여 분류기를 훈련
 - a. 훈련 데이터에 있는 이미지와 레이블을 연관시킬 수 있는 모델 구현 함수 표현
3. 새로운 이미지 분류기를 평가
 1. 모델과 몇 개의 테스트 이미지를 입력받은 PREDICT라는 함수 구현

Machine Learning: Data-Driven Approach

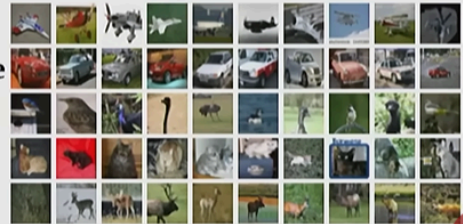
1. Collect a dataset of images and labels
2. Use Machine Learning algorithms to train a classifier
3. Evaluate the classifier on new images

Example training set

```
def train(images, labels):  
    # Machine learning!  
    return model
```

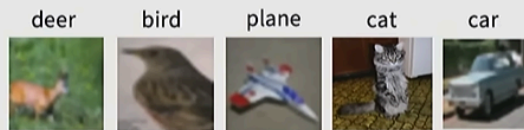
```
def predict(model, test_images):  
    # Use model to predict labels  
    return test_labels
```

airplane
automobile
bird
cat
deer

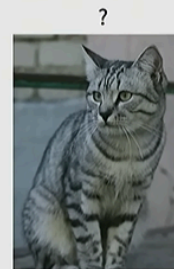


최근접 분류기

First classifier: Nearest Neighbor

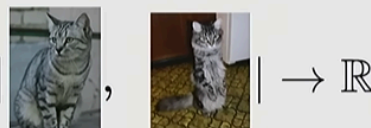


Training data with labels



query data

Distance Metric



5개의 이미지가 훈련 데이터에 있다고 가정하면 쿼리 이미지가 예측을 위한 이미지.

훈련 데이터 중에서 어떤 데이터가 쿼리 이미지와 유사한지 확인하는 것

두 이미지 쌍을 입력으로 받아 비교하고 유사도를 나타내는 값을 반환

그 유사도 반환 방법 중 가장 널리 쓰이는 것은 L1 거리

= I1, I2 사이의 픽셀 차이의 절댓값을 모두 더한 값으로 정의

Distance Metric to compare images

L1 distance:
$$d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$$

test image				training image				pixel-wise absolute value differences			
56	32	10	18	10	20	24	17	46	12	14	1
90	23	128	133	8	10	89	100	82	13	39	33
24	26	178	200	12	16	178	170	12	10	0	30
2	0	255	220	4	32	233	112	2	32	22	108

-

=

add → 456

픽셀 단위로 두 이미지 사이의 차이를 계산하고 모두 더한다

값이 두 이미지 사이의 거리로 정의된다

이게 가장 기본적인 거리 계산 함수

(구현 방법)

```
import numpy as np

class NearestNeighbor:
    def __init__(self):
        pass
```

Nearest Neighbor classifier

```
def train(self, X, y):
    """ X is N x D where each row is an example. Y is 1-dimension of size N """
    # the nearest neighbor classifier simply remembers all the training data
    self.Xtr = X
    self.ytr = y
```

Memorize training data

1단계 : 훈련 데이터 암기

학습 함수는 데이터를 메모리에 저장한다

```
# loop over all test rows
for i in xrange(num_test):
    # find the nearest training image to the i'th test image
    # using the L1 distance (sum of absolute value differences)
    distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
    min_index = np.argmin(distances) # get the index with smallest distance
    Ypred[i] = self.ytr[min_index] # predict the label of the nearest example
```

For each test image:
Find closest train image
Predict label of nearest image

2단계 : 예측 함수

파이썬 라이브러리와 넘파이 등을 이용하여 예측을 수행

샘플과 훈련 데이터 사이의 거리 계산, 테스트 샘플에 대해 최솟값을 취한 다음 평균 인덱스를 가진 샘플에 대해 최솟값에 대한 레이블로 출력

Q. n개의 훈련 데이터를 통해 얼마나 빠르게 훈련과 예측이 이루어지나?

A.

훈련 단계에서는 $O(1)$. 데이터를 이동하지 않고 1차 연산 없이 훈련 단계를 완료할 수 있다
예측 단계에서는 훈련 데이터가 n 개라면 모든 테스트 이미지와 훈련 데이터에 있는 이미지 사이의 거리 계산이 필요 = $O(n)$

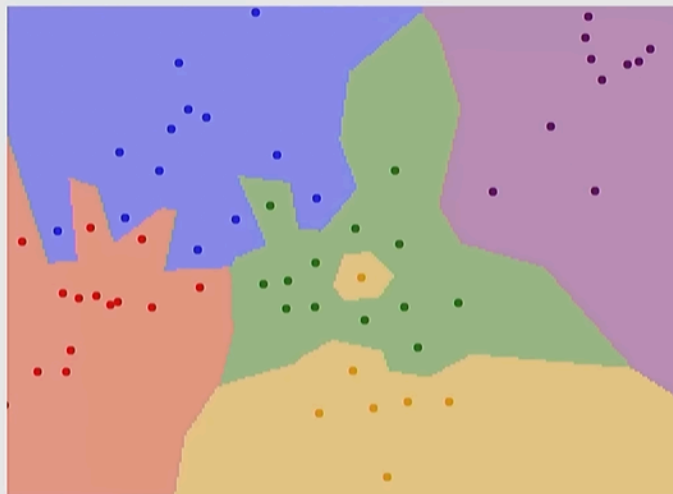
테스트 단계나 예측 단계에서 데이터와 각 데이터 포인트, 예제 비교에 너무 많은 시간을 소비함

우리는 예측 속도가 빠른 분류기 구축을 원한다. 차라리 훈련 중에 시간이 많이 걸리는 것이 더 좋다. 그것은 오프라인에서 가능하기 때문이다.

최근접 알고리즘

= 공간을 분할함

What does this look like?



이 이미지가 뭐로 보이니

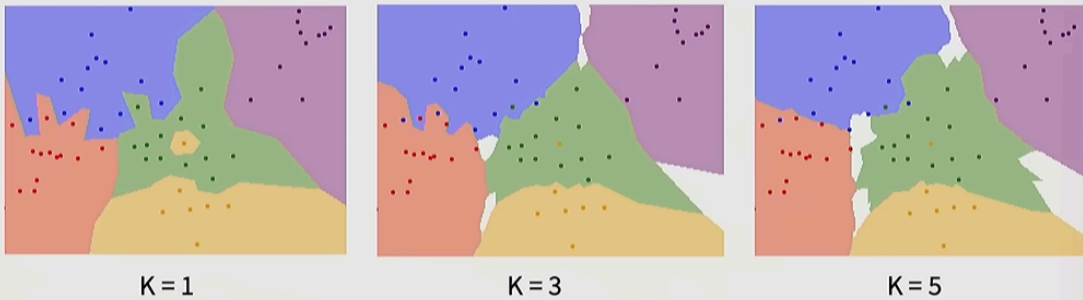
= 최근접 이웃 알고리즘. 공간을 분할한다. 근데 이 예시는 노란색이 초록 가운데 있다. 이게 노이즈, 우리는 이런 노이즈를 해결해야 함

그 이유는 저 노란 점이 단 하나의 점이기 때문에 가장 가까운 이웃 하나만을 사용하기 때문에 이런 문제 발생

가장 가까운 이웃의 수를 늘리면 최근접 이웃 알고리즘이 k-최근접 알고리즘으로 바뀜

K-Nearest Neighbors

Instead of copying label from nearest neighbor,
take majority vote from K closest points



하나 이상의 샘플을 선택, 종종 주어진 테스트 이미지에 대한 레이블 식별을 위해 다수결 투표 활용

흰색 영역이 생기는 문제 발생. 이거는 세 클래스에서 동일한 수의 샘플을 추출했기 때문에 완전한 결론을 내릴 수 없는 영역. = 식별 불가능.

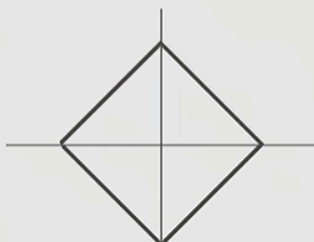
이러한 유형의 공간을 볼 때 이 영역이 더 많은 데이터를 수집하기에 좋은 공간, 불분명한 공간, = 데이터 수집에 중요한 지역

L1 거리를 시각화하면 거리 함수는 이런 식으로 나타난다.

K-Nearest Neighbors: Distance Metric

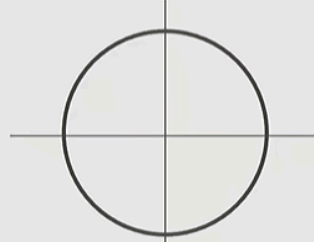
L1 (Manhattan) distance

$$d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$$



L2 (Euclidean) distance

$$d_2(I_1, I_2) = \sqrt{\sum_p (I_1^p - I_2^p)^2}$$



정사각형 위의 모든 점들은 중심점으로부터 원점까지 거리가 같다. 이것은 L1 거리 함수가 어떻게 작동하는지 이해하는 좋은 방법

L2는 절댓값 대신 차이의 제곱을 계산하여 합산. (= 제곱근) 시각화하면 원이 나타남

역시 원의 중심으로부터 같은 거리에 위치

회전하면 L1은 다른 프레임워크와 다른 값을 갖지만 L2는 변화없음

정보를 보존하고 싶을 때는 L1. 특징을 기반으로 거리를 보존하고 강화
임의적인 특징이면 L2가 적합.

축을 회전하면 거리와 함수가 바뀌는데 L2는 그대로이다.

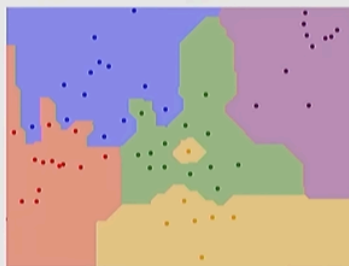
= L1은 특징 값에 민감하다

L1과 L2를 사용하여 공간을 다시 시각화한다면

K-Nearest Neighbors: Distance Metric

L1 (Manhattan) distance

$$d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$$



K = 1

L2 (Euclidean) distance

$$d_2(I_1, I_2) = \sqrt{\sum_p (I_1^p - I_2^p)^2}$$



K = 1

k가 1일 때 가장 가까운 이웃이 L1, L2인 경우 공간 분할 발생

L1 함수의 경우 대부분의 경계가 두 축과 평행. 특징에 민감한 L1 특성 반영. 경계 분리가 좀 더 부드럽게 이루어짐

Q. 왜 가장 가까운 이웃에 대해 이야기하는 걸까

= 가장 쉬운 해결책, 하이퍼파라미터라는 주제를 살펴볼 수 있다

하이퍼파라미터 : 알고리즘을 실행하기 위해 결정해야 하는 변수 중 일부

k 값을 하이퍼파라미터로 정의

가장 가까운 이웃을 몇 개로 정의하느냐에 따라 결과가 달라진다

= 데이터 셋에 따라, 문제에 따라 달라지기 때문에 각 문제에 맞게 최적화할 수 있는 방법을 찾아야 하는데 그게 머신러닝 알고리즘이나 딥러닝 알고리즘을 통한 하이퍼파라미터 튜닝

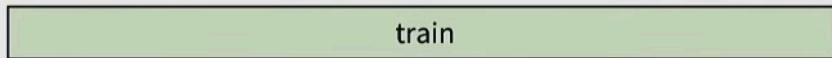
- 하이퍼 파라미터 선택 방법 :

훈련 데이터에 가장 적합한 파라미터 선택,

Setting Hyperparameters

Idea #1: Choose hyperparameters that work best on the training data

BAD: $K = 1$ always works perfectly on training data



Idea #2: choose hyperparameters that work best on test data

BAD: No idea how algorithm will perform on new data



Idea #3: Split data into train, val; choose hyperparameters on val and evaluate on test

Better!



Stanford

1. 훈련 데이터에 가장 적합한 하이퍼파라미터 선택

= 최소 훈련 손실을 만드는 값을 고르는 것. 훈련 데이터에서는 잘 되지만 좋은 방법이 아니다

→

= 특히 최근접 이웃에서는 $K = 1$ 이 항상 최고. 훈련 데이터를 암기하기 때문에 항상 정확도 100%

2. 테스트 데이터에 가장 적합한 하이퍼파라미터 선택

= 1번보다는 낫지만 큰 문제가 있다. 테스트 데이터에서 가장 잘 되는 값을 찾는 것이기 때문에 일종의 부정행위(cheating)

= 테스트 셋에 없는 새로운 데이터에서 모델이 어떻게 작동할지 알 수 없다 = 일반화 성능을 모름

3. 훈련 데이터 일부를 검증(validation) 셋으로 떼어내기

= train으로 학습 → validation에서 하이퍼파라미터 최적화 → 찾은 값으로 test 셋 예측

= 훨씬 좋은 방법. 하지만 validation 셋은 보통 훨씬 작아서 전체 데이터를 잘 대표하지 못할 수도 있다

Setting Hyperparameters

train

Idea #4: Cross-Validation: Split data into folds, try each fold as validation and average the results

fold 1	fold 2	fold 3	fold 4	fold 5
fold 1	fold 2	fold 3	fold 4	fold 5
fold 1	fold 2	fold 3	fold 4	fold 5
fold 1	fold 2	fold 3	fold 4	fold 5
fold 1	fold 2	fold 3	fold 4	fold 5

test

Useful for small datasets, but not used too frequently in deep learning

Stanford

1. 교차 검증 (Cross-Validation)

= 훈련 데이터를 여러 개의 fold로 나눈다 (여기서는 5개)

= 각 fold가 한 번씩 validation 셋 역할 → 5번 반복 (5-fold cross validation) → 정확도 평균

= 하이퍼파라미터 값을 하나 정하고 5번 돌려서 평균 정확도 계산, 이걸 여러 값에 대해 반복해서 최적값 찾기 → 그 값을 테스트 셋에 적용

= 더 신뢰할 수 있고 결과도 좋다

= 하지만 대규모 딥러닝에서는 거대한 데이터셋으로 5번씩 반복하는 게 너무 어려워서 잘 안 씀. 직관으로 정하거나 validation 셋 하나만 쓰는 경우가 많다

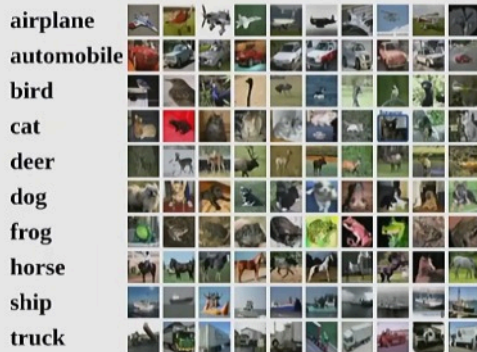
= 대규모 데이터가 아닌 분야의 연구 논문에서는 결과 재현성을 위해 이런 교차 검증을 요구하는 경우가 많다

→

CIFAR10 데이터셋

Example Dataset: CIFAR10

10 classes
50,000 training images
10,000 testing images



Alex Krizhevsky, "Learning Multiple Layers of Features from Tiny Images", Technical Report, 2009.

Test images and nearest neighbors

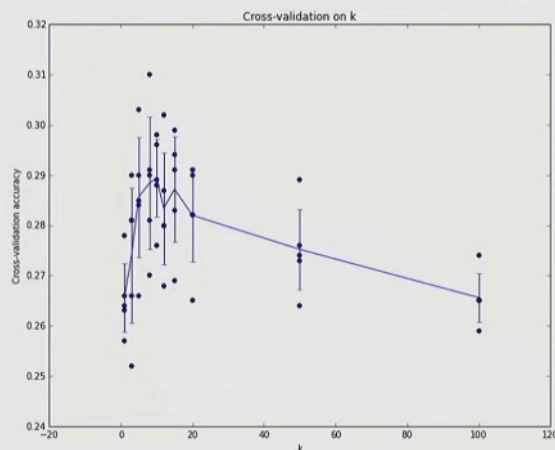


Stanford

과제에서 자주 사용할 데이터셋. 10개 클래스, 훈련 이미지 50,000장, 테스트 이미지 10,000장

오른쪽은 각 테스트 이미지에 대해 최근접 이웃을 실행해서 가장 가까운 이웃 10개를 시각화한 것

Setting Hyperparameters



Example of
5-fold cross-validation
for the value of k.

Each point: single
outcome.

The line goes
through the mean, bars
indicated standard
deviation

(Seems that $k \approx 7$ works best
for this data)

Stanford

K 값을 몇으로 해야 하나? = 5-fold 교차 검증으로 실험

각 점이 5-fold 중 하나의 결과, 선은 평균, 막대는 표준편차

= K = 7일 때 가장 좋다. 정확도 약 28~29%

= 10개 클래스 분류 문제에서 무작위로 찍으면 10%이기 때문에 이 정도면 나쁘지 않음. 작동은 하지만 개선할 여지가 많다

다시 예시를 보면 틀린 게 많다. 4번째 줄은 개구리인데 가장 가까운 첫 번째 이웃은 개

= 거리를 픽셀에 적용하기 때문. 픽셀 단위로 보면 대부분의 픽셀이 비슷한 색이라 더 가깝게 계산된다

→

k-Nearest Neighbor with pixel distance never used.

- Distance metrics on pixels are not informative

(All three images on the right have the same pixel distances to the one on the left)

Stanford

= 픽셀 값에 대한 거리는 좋은 선택이 아니다. 실제로 절대 쓰지 않음. 더 좋은 방법은 이후 강의에서

원본 이미지와 오른쪽 세 이미지(가림, 1픽셀 이동, 색조 변경)는 색이나 가려짐 정도가 달라 보이지만 원본과의 픽셀 거리가 모두 같다

세 번째(Shifted)는 같은 이미지를 오른쪽으로 1픽셀 옮긴 것뿐이라 사람 눈에는 전혀 차이가 없지만, 원본과의 거리는 다른 두 이미지와 같다

Q. (동점일 때) 어떻게 결정하나?

A. 그런 경우에는 보통 상위 후보 중 하나를 무작위로 선택

= 유전학이나 의료 영상 같은 문제를 풀 때 최근접 이웃 공간을 시각화했는데 좋은 샘플이 없거나 모호한 공간이 보이면, 그 영역에 해당하는 샘플을 더 찾아서 수집한다

K-Nearest Neighbors: Summary

In image classification we start with a training set of images and labels, and must predict labels on the test set

The K-Nearest Neighbors classifier predicts labels based on the **K nearest training examples**

Distance metric and K are hyperparameters

Choose hyperparameters using the validation set

Only run on the test set once at the very end!

Stanford

K 최근접 이웃 정리

= 가장 쉬운 데이터 기반 알고리즘

= 이미지 분류에서는 이미지와 레이블로 된 훈련 셋으로 시작해서 테스트 셋의 레이블을 예측해야 한다

= K 최근접 이웃 분류기는 가장 가까운 K개의 훈련 예제를 기반으로 레이블 예측

= 거리 척도와 K는 하이퍼파라미터이고 결과에 매우 중요한 역할

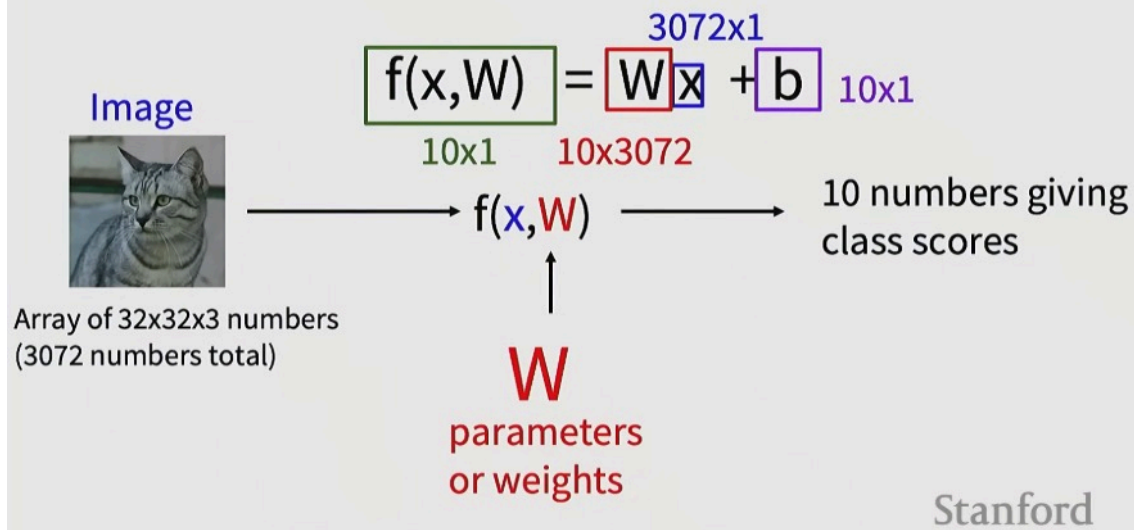
= 하이퍼파라미터는 validation 셋으로 선택

= 테스트 셋은 맨 마지막에 딱 한 번만 실행!

선형 분류기

= 거의 모든 딥러닝의 가장 중요한 구성 요소

Parametric Approach: Linear Classifier



최근점 이웃과의 차이 = 파라메트릭 접근 방식

= 입력 이미지를 출력 클래스(숫자)로 매핑하는 파라미터 W (가중치)를 학습해서 찾는다

= 함수 f 의 출력은 보통 이미지가 10개의 클래스 각각에 얼마나 속하는지를 나타내는 점수

이미지는 $32 * 32 * 3$ 배열 = 3072개의 숫자 → x 는 $3072 * 1$ 벡터

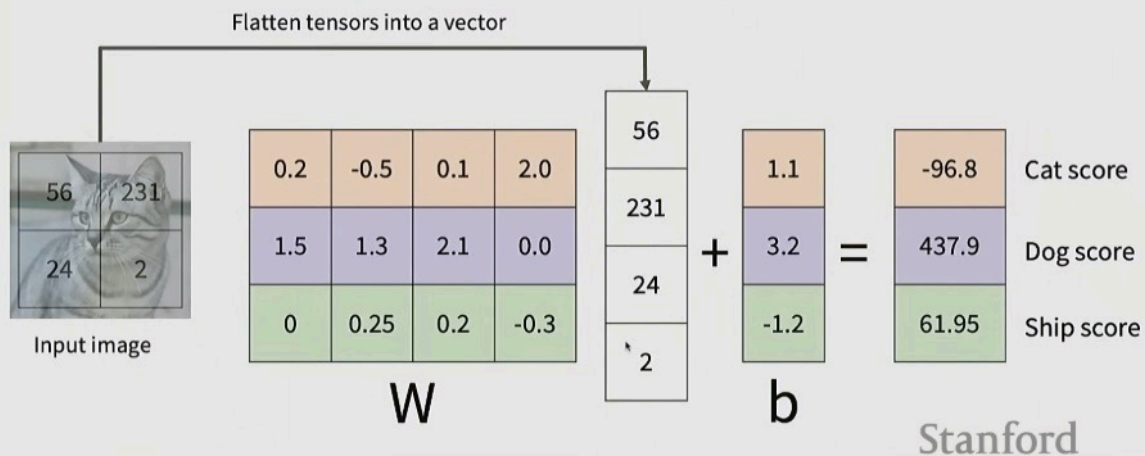
출력 클래스가 10개 → 점수도 10개 필요 → 출력은 $10 * 1$ 벡터

= W 는 $10 * 3072$ 행렬

선형 함수를 완성하기 위해 편향(bias) b 도 사용. 입력과 무관한 값으로, 클래스 점수를 이동시켜서 각 클래스를 더 잘 분리하게 해 준다

이런 선형 함수들을 하나씩 이어 붙이면 큰 신경망이 된다. 유명한 신경망 구조들을 보면 선형 함수가 어디에나 있다

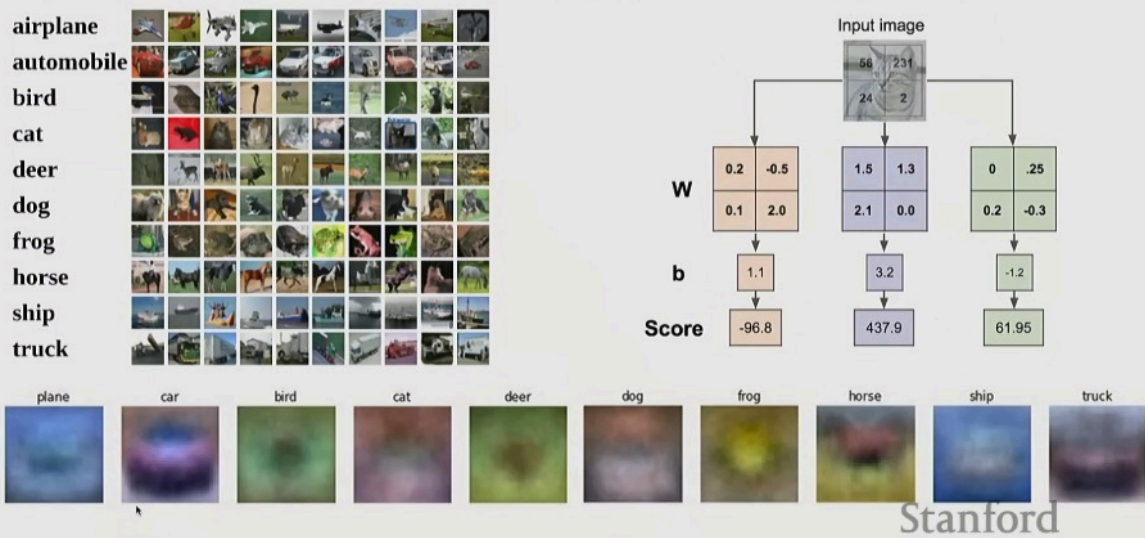
Example with an image with 4 pixels, and 3 classes (cat/dog/ship) Algebraic Viewpoint



대수적 관점

- = 더 단순하게 픽셀 4개짜리(2 * 2) 이미지로 예시. 클래스는 고양이, 개, 배 3개
- = 이미지를 벡터로 펴서 W를 곱하고 b를 더하면 각 클래스의 점수가 나온다
- = 이 예시에서는 개 점수(437.9)가 가장 높다 = W가 아직 좋지 않다

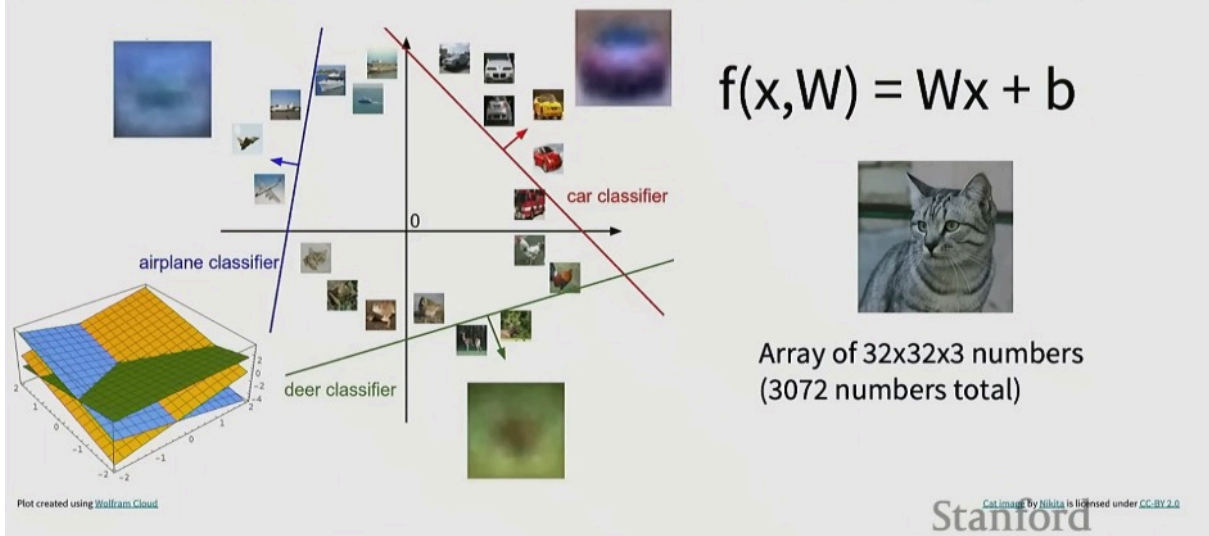
Interpreting a Linear Classifier: Visual Viewpoint



시각적 관점

- = 행렬 W의 각 행이 해당 클래스의 템플릿
- = CIFAR10으로 학습한 후 10개 클래스의 템플릿을 이미지로 보면 흥미롭게도 car는 자동차 앞모습 같은 템플릿이 보인다. 선형 분류기 하나만으로 이렇게 된다

Interpreting a Linear Classifier: Geometric Viewpoint



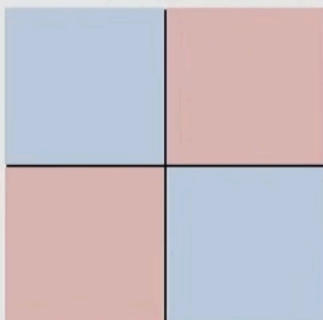
기하학적 관점

- = 2D 공간에서는 각 클래스를 다른 클래스와 분리하는 선을 찾는 것
- = 고차원 공간에서는 선 대신 초평면(hyperplane)
- = bias가 없으면 모든 선이 원점을 지나야 해서 말이 안 된다. bias가 있어야 더 신뢰할 수 있는 결정 경계를 만들 수 있다

Hard cases for a linear classifier

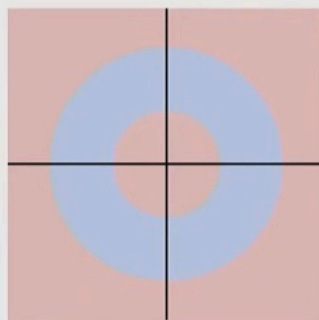
Class 1:
First and third quadrants

Class 2:
Second and fourth quadrants



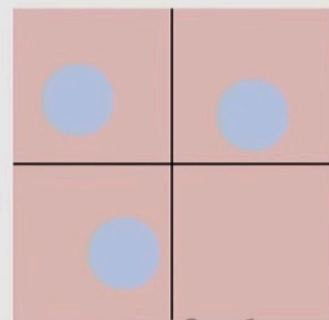
Class 1:
 $1 \leq L2 \text{ norm} \leq 2$

Class 2:
Everything else



Class 1:
Three modes

Class 2:
Everything else



Stanford

선형 분류기가 어려운 경우

- = 선형 분류기는 유용하고 복잡한 신경망의 구성 요소지만 선형으로 분리할 수 없는 데이터가 많다

1. 클래스 1이 1, 3사분면이고 클래스 2가 2, 4사분면이면 선형으로 분리할 방법이 없다
 2. 원점으로부터 거리가 1~2 사이면 클래스 1, 나머지는 클래스 2인 경우
 3. 공간 속 세 영역(3개의 모드)이 하나의 클래스이고 나머지는 전부 다른 클래스인 경우
- = 이런 경우 모두 분리가 매우 어렵다

손실 함수

Suppose: 3 training examples, 3 classes.
With some W the scores $f(x, W) = Wx$



cat	3.2	1.3	2.2
car	5.1	4.9	2.5
frog	-1.7	2.0	-3.1

A loss function tells how good our current classifier is

Given a dataset of examples $\{(x_i, y_i)\}_{i=1}^N$

Where x_i is image and y_i is (integer) label

Loss over the dataset is a average of loss over examples:

$$L = \frac{1}{N} \sum_i L_i(f(x_i, W), y_i)$$

Stanford

남은 문제 = 각 이미지를 클래스별 점수로 매핑하는 W 를 어떻게 고를까

1. 손실 함수(목적 함수)를 정의. 분류기(모델)가 얼마나 잘못 작동하고 있는지를 수치화 = 훈련 데이터 점수에 대한 불만족 정도
2. 그 손실을 최소화하도록 W 값을 효율적으로 바꾸는 방법 찾기 = 최적화 (다음 강의 주제)

단순하게 고양이, 자동차, 개구리 3개 클래스로 예시

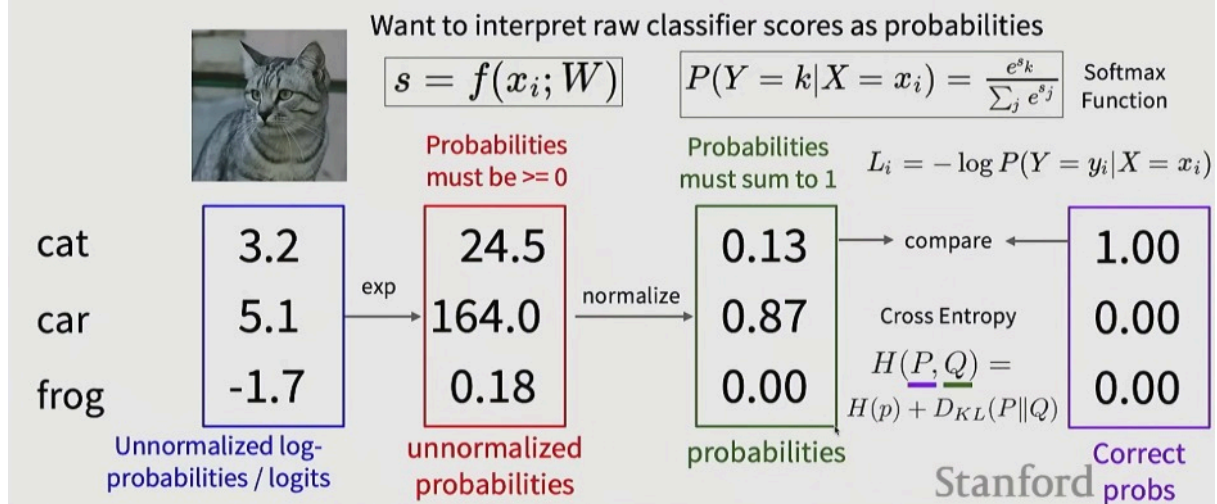
입력 이미지 x_i 와 레이블 y_i 로 문제를 파라미터화

= 예측 점수 $f(x_i, W)$ 와 정답 y_i 를 비교해서 점수가 얼마나 나쁜지 보는 손실 함수 L_i

= 보통 샘플 수로 나눠서 평균을 낸다

Softmax 분류기

Softmax Classifier (Multinomial Logistic Regression)



고양이 이미지의 점수는 3.2, 5.1, -1.7

이 점수는 선형 함수의 출력이라 범위가 정해져 있지 않고 다루기 어렵다 = 확률로 바꾸는 것이 가장 좋다

= 입력 이미지 x_i 가 클래스 k 일 확률

1. 점수에 지수 함수(exp)를 적용 = 출력이 항상 양수 (확률은 항상 양수여야 하니까)
2. 전체 합으로 나눠서 정규화 = 합이 1인 확률 분포

= 이 W 는 이 이미지가 고양이일 확률을 13%로 생각한다. W 가 좋은 값이 아니라서 틀린 것
= 최적화해서 바꿔야 한다

정규화 전 점수 = 로짓(logits, 정규화되지 않은 로그 확률)

= 로지스틱 회귀와 같은 프레임워크. 클래스가 여러 개라서 다항 로지스틱 회귀 (Multinomial Logistic Regression)

손실 함수 L 정의

= 목표 : 샘플이 정답 클래스에 속할 확률을 최대화 (여기서는 0.13을 최대화)

= 최대화 문제를 최소화 문제로 바꾸려고 음수를 취하고, 숫자를 다루기 쉽게 로그를 취한다

= 정답 클래스 확률의 $-\log$ = 손실 함수. 다른 수업에서는 최대우도추정(MLE)이라고도 부르는 같은 알고리즘

다른 해석

= 추정한 확률 분포와 정답 확률 분포를 일치시키고 싶다 = KL divergence(쿨백-라이블러 발산) 최소화. 정보 이론 관점

= 이 경우 KL divergence도 같은 $-\log$ 식으로 정리된다

= 교차 엔트로피 $H(P, Q) = H(P) + \text{KL divergence}$. 정답을 원-핫 인코딩하면 $H(P) = 0$ 이라서 역시 같은 $-\log$ 식

= 그래서 이 함수를 교차 엔트로피(cross entropy) 손실이라고 부른다

Softmax Classifier (Multinomial Logistic Regression)



Want to interpret raw classifier scores as probabilities

$s = f(x_i; W)$

$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$

Softmax Function

Maximize probability of correct class Putting it all together:

$L_i = -\log P(Y = y_i|X = x_i)$
 $L_i = -\log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$

cat	3.2
car	5.1
frog	-1.7

Q2: At initialization all s will be approximately equal; what is the loss?
 A: $-\log(1/C) = \log(C)$,
 If $C = 10$, then $L_i = \log(10) \approx 2.3$

Stanford

Q1. 손실 L_i 의 최솟값과 최댓값은?

A. 최솟값 0 (정답 확률 1), 최댓값 무한대 (정답 확률이 0에 가까워지면 $\log$ 가 $-\infty$ 로 가는데 앞에 -가 붙으니까)

Q2. 초기화할 때는 W 가 거의 무작위라 모든 점수가 비슷하다. 클래스가 C 개일 때 softmax 손실 L_i 는?

A. 모든 확률이 약 $1/C$ $= -\log(1/C) = \log(C)$. $C = 10$ 이면 $\ln(10) \approx 2.3$